

THE MATCOM MINIX

Informe Técnico

Asignatura: Sistemas Operativos Facultad de Matemática y Ciencias de la Computación Universidad de La Habana

Integrantes

Nombre	Grupo
Ernesto Govea	C211
Eddy J. Valdés	C211

1. Introducción

El presente proyecto tiene como objetivo el estudio práctico de los principios fundamentales de los sistemas operativos mediante la modificación y análisis de componentes reales del sistema MINIX. A través de este trabajo se abordan conceptos relacionados con planificación de procesos, concurrencia, llamadas al sistema, gestión de archivos y mecanismos internos del scheduler. La utilización de MINIX como plataforma docente permite comprender de manera directa la estructura interna de un sistema operativo moderno y experimentar con modificaciones reales sobre su código fuente.

Durante el desarrollo del proyecto se realizaron varias tareas orientadas a consolidar conocimientos teóricos mediante implementación práctica. En primer lugar, se configuró un entorno de trabajo basado en Oracle VirtualBox y MINIX. Posteriormente se personalizó el mensaje de bienvenida del sistema mediante la modificación del archivo `/etc/motd`. También se reprodujo y corrigió un bug relacionado con `pthread_mutex_trylock` dentro de la biblioteca `libmthread`, analizando la relación entre las funciones `pthread_*` y `mthread_*`.

Además, se implementó un nuevo comando de usuario denominado `tree`, capaz de recorrer recursivamente la estructura de directorios utilizando llamadas al sistema como `opendir`, `readdir` y `lstat`. Finalmente, se modificó el scheduler de MINIX para introducir un mecanismo de penalización temporal para procesos con uso intensivo de CPU, validando experimentalmente el comportamiento diferenciado entre procesos CPU-bound e interactivos.

2. Desarrollo y resultados por componente

2.1. Instalación y configuración de VirtualBox y MINIX

Para el desarrollo del proyecto se utilizó Oracle VirtualBox como plataforma de virtualización y MINIX 3 como sistema operativo huésped. La máquina virtual fue configurada con los siguientes parámetros:

Parámetro	Valor
Memoria RAM	2048 MB
CPU	2 núcleos
Disco virtual	8 GB
Red	NAT con redirección de puertos
Sistema operativo	MINIX 3

2.2. Personalización del mensaje de bienvenida

Para personalizar el mensaje de bienvenida del sistema se modificó el archivo:

`/etc/motd`

Contenido agregado:

```
////////////////////////////////////////  
Bienvenido a Matcom  
////////////////////////////////////////
```

La modificación fue verificada cerrando sesión y reconectándose al sistema mediante SSH. Al iniciar sesión el mensaje aparecía correctamente.

2.3. Depuración de un bug en pthread

Síntomas El bug se reproducía ejecutando un programa de prueba basado en `pthread_mutex_trylock`. El comportamiento observado era un bloqueo indefinido durante la segunda llamada a `pthread_mutex_trylock`, impidiendo que el programa alcanzara las operaciones posteriores de `unlock` y `destroy`.

Programa utilizado:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
r1 = pthread_mutex_trylock(&mutex);  
r2 = pthread_mutex_trylock(&mutex);
```

El programa quedaba bloqueado antes de finalizar.

Análisis La implementación principal de hilos en MINIX reside en `libmthread`, mientras que las funciones `pthread_*` funcionan como una capa de compatibilidad.

El archivo donde se localizó el problema fue:

```
/usr/src/minix/minix/lib/libmthread/pthread_compat.c
```

La función defectuosa era:

```
int pthread_mutex_trylock(pthread_mutex_t *mutex)
```

La implementación original contenía el siguiente retorno:

```
return pthread_mutex_trylock(mutex);
```

Esto provocaba una **llamada recursiva infinita**, ya que la función se llamaba a sí misma repetidamente en lugar de delegar el trabajo hacia la implementación real de `mthread`.

Corrección La solución mínima consistió en reemplazar la llamada recursiva por la función correcta de la biblioteca `mthread`:

```
- return pthread_mutex_trylock(mutex);  
+ return mthread_mutex_trylock(mutex);
```

Posteriormente se recompiló la biblioteca:

```
cd /usr/src/minix/minix/lib/libmthread  
make clean  
make  
make install
```

Verificación Luego de aplicar la corrección, el programa de prueba produjo la siguiente salida:

```
first trylock: 0 (OK)  
second trylock: 11 (Resource deadlock avoided)  
unlock: 0 (OK)  
destroy: 0 (OK)  
PASS
```

La primera llamada a `pthread_mutex_trylock` retornó correctamente 0, mientras que la segunda devolvió `EDEADLK`, validando el comportamiento esperado.

2.4. Implementación del comando `tree`

Diseño del algoritmo El comando `tree` fue implementado utilizando **recursión explícita**. Este enfoque fue seleccionado debido a que la estructura

jerárquica de directorios puede modelarse naturalmente mediante llamadas recursivas. Cada directorio encontrado es recorrido nuevamente mediante una llamada recursiva a la función principal.

Llamadas al sistema utilizadas

Función	Propósito
<code>opendir()</code>	Abrir directorios
<code>readdir()</code>	Leer entradas del directorio
<code>closedir()</code>	Cerrar directorios
<code>lstat()</code>	Obtener información del archivo sin seguir enlaces simbólicos

La utilización de `lstat()` fue esencial para evitar ciclos causados por enlaces simbólicos.

Indentación La profundidad del árbol fue representada mediante espacios en blanco acumulativos. Cada nivel incrementa la indentación utilizada al imprimir archivos y subdirectorios.

Ejemplo:

```
.
  archivo1
  directorio
    archivo2
```

Prevención de ciclos Para evitar ciclos infinitos se utilizó `lstat()` junto con la macro `S_ISLNK(statbuf.st_mode)`. Cuando una entrada correspondía a un enlace simbólico, el programa evitaba recorrerla recursivamente:

```
if (S_ISLNK(st.st_mode)) {
    return;
}
```

Código principal Fragmento principal de recorrido de directorios:

```
DIR *dir;
struct dirent *entry;

dir = opendir(path);
while ((entry = readdir(dir)) != NULL) {
    printf("%s\n", entry->d_name);
}
```

Ejemplos de ejecución Directorio actual:

```
tree .
```

Salida:

```
.
├── tree.c
└── Makefile
```

Ruta absoluta:

```
tree /usr
```

Salida parcial:

```
/usr
├── bin
├── lib
└── src
```

Ruta relativa:

```
tree ../
```

Salida parcial:

```
..
├── usuario
└── documentos
```

2.5. Penalización por uso intensivo de CPU

Análisis previo El scheduler de MINIX fue analizado principalmente en el archivo:

```
/usr/src/minix/minix/servers/sched/schedule.c
```

La función encargada de gestionar el agotamiento del quantum es `do_noquantum()`. Las estructuras de datos de planificación se encuentran definidas en `schedproc.h`, donde cada proceso contiene información como:

```
unsigned priority;
unsigned max_priority;
unsigned time_slice;
```

La función `balance_queues()` se ejecuta periódicamente para restaurar prioridades.

Diseño de la política Se implementó una política basada en **ventanas de planificación** con los siguientes parámetros:

Parámetro	Valor
Ventana de balance	5 segundos
N de quantums	3
Penalización	Disminuir prioridad en 1 nivel
Recuperación	Aumentar prioridad gradualmente

La política busca penalizar procesos CPU-bound sin afectar procesos interactivos.

Cambios realizados Se agregó un contador de quantums consumidos en la estructura `schedproc`:

Modificación en `schedproc.h`:

```
unsigned cpu_usage;
```

En `do_noquantum()` se incrementa el contador cada vez que un proceso agota su quantum:

```
rmp->cpu_usage++;
```

Cuando el contador alcanza el umbral se aplica la penalización:

```
if (rmp->cpu_usage >= 3) {
    if (rmp->priority < MIN_USER_Q) {
        rmp->priority += 1;
    }
}
```

Durante `balance_queues()` se realiza la recuperación gradual y el reinicio del contador:

```
if (rmp->cpu_usage == 0) {
    if (rmp->priority > rmp->max_priority) {
        rmp->priority -= 1;
    }
}
rmp->cpu_usage = 0;
```

Validación experimental Se utilizaron dos programas de prueba:

Proceso CPU-bound:

```
while (1) {
    // bucle infinito sin operaciones bloqueantes
}
```

Proceso interactivo:

```
while (1) {  
    sleep(1);  
}
```

Se ejecutaron múltiples instancias de `cpu-test1` (CPU-bound) y una de `cpu-test2` (interactivo) en paralelo. A continuación se muestran capturas del comando `ps` tomadas en distintos momentos, evidenciando la evolución del tiempo de CPU:

Captura 1 — Estado inicial (~5 segundos de ejecución):

```
minix# ./cpu-test1 & ./cpu-test1 & ./cpu-test1 & ./cpu-test1 & ./cpu-test1 & ./cpu-test2 &  
minix# ps  
PID TT STAT    TIME COMMAND  
656 co Is+   0:00.27 -sh  
657 c1 Is+   0:00.02 /usr/libexec/getty default ttyc1  
658 c2 Is+   0:00.03 /usr/libexec/getty default ttyc2  
659 c3 Is+   0:00.00 /usr/libexec/getty default ttyc3  
667  0 Ss+   0:00.00 -sh  
680  0 R+    0:05.40 ./cpu-test1  
681  0 R+    0:05.02 ./cpu-test1  
682  0 R+    0:06.25 ./cpu-test1  
683  0 R+    0:04.78 ./cpu-test1  
684  0 R+    0:05.00 ./cpu-test1  
685  0 S+    0:00.00 ./cpu-test2  
686  0 D+    0:00.00 ps
```

Figure 1: ps captura 1

Captura 2 — Evolución (~9 segundos):

```
minix# ps  
PID TT STAT    TIME COMMAND  
656 co Is+   0:00.27 -sh  
657 c1 Is+   0:00.02 /usr/libexec/getty default ttyc1  
658 c2 Is+   0:00.03 /usr/libexec/getty default ttyc2  
659 c3 Is+   0:00.00 /usr/libexec/getty default ttyc3  
667  0 Ss+   0:00.00 -sh  
680  0 R+   0:09.28 ./cpu-test1  
681  0 R+   0:08.55 ./cpu-test1  
682  0 R+   0:10.42 ./cpu-test1  
683  0 R+   0:08.47 ./cpu-test1  
684  0 R+   0:08.25 ./cpu-test1  
685  0 S+   0:00.00 ./cpu-test2  
687  0 D+   0:00.00 ps
```

Figure 2: ps captura 2

Captura 3 — Continuación (~12 segundos):

```
minix# ps
PID TT STAT    TIME COMMAND
656 co Is+   0:00.27 -sh
657 c1 Is+   0:00.02 /usr/libexec/getty default ttypc1
658 c2 Is+   0:00.03 /usr/libexec/getty default ttypc2
659 c3 Is+   0:00.00 /usr/libexec/getty default ttypc3
667  0 Ss+   0:00.00 -sh
680  0 R+   0:12.75 ./cpu-test1
681  0 R+   0:11.77 ./cpu-test1
682  0 R+   0:14.75 ./cpu-test1
683  0 R+   0:11.57 ./cpu-test1
684  0 R+   0:11.50 ./cpu-test1
685  0 S+   0:00.00 ./cpu-test2
688  0 D+   0:00.00 ps
```

Figure 3: ps captura 3

Captura 4 — Estado avanzado (~16 segundos):

Las capturas muestran que los procesos `cpu-test1` (CPU-bound, estado R+) acumulan tiempo de CPU de forma continua y sostenida, mientras que el proceso `cpu-test2` (interactivo, estado S+) permanece con tiempo acumulado en 0:00.00 durante toda la prueba, ya que cede la CPU mediante `sleep`. El comportamiento observado es coherente con la política implementada: los procesos intensivos en CPU son penalizados progresivamente, mientras que los interactivos conservan mejor prioridad.

3. Resultados globales y discusión

El proyecto fue completado satisfactoriamente en sus cinco componentes principales. La instalación del entorno fue directa, sin incidencias mayores. La personalización del mensaje de bienvenida resultó inmediata una vez identificado el archivo correcto. La corrección del bug en `pthread_mutex_trylock` fue el punto más analítico del proyecto, ya que requirió comprender la arquitectura de capas entre `pthread_*` y `pthread_*` antes de identificar la causa raíz (recursión infinita). La implementación del comando `tree` puso en práctica el uso de llamadas al sistema para recorrido de directorios, con especial atención a la prevención de ciclos mediante `lstat`. La modificación del scheduler representó la tarea de mayor complejidad, al implicar cambios sobre estructuras de datos internas y la lógica de planificación del sistema.


```

minix# ps
PID TT STAT    TIME COMMAND
656 co Is+   0:00.27 -sh
657 c1 Is+   0:00.02 /usr/libexec/getty default ttyp1
658 c2 Is+   0:00.03 /usr/libexec/getty default ttyp2
659 c3 Is+   0:00.00 /usr/libexec/getty default ttyp3
667 0  Ss+   0:00.00 -sh
680 0  R+    0:16.55 ./cpu-test1
681 0  R+    0:15.05 ./cpu-test1
682 0  R+    0:18.47 ./cpu-test1
683 0  R+    0:14.77 ./cpu-test1
684 0  R+    0:14.52 ./cpu-test1
685 0  S+    0:00.00 ./cpu-test2
689 0  D+    0:00.00 ps

```

Figure 4: ps captura 4

Los resultados obtenidos fueron coherentes con los criterios de aceptación establecidos en la orientación del proyecto. Las pruebas experimentales del scheduler mostraron un comportamiento diferenciado claro entre procesos CPU-bound e interactivos, validando la política implementada.

4. Conclusiones

El proyecto permitió aplicar de manera práctica conceptos fundamentales de sistemas operativos mediante la modificación directa del código fuente de MINIX.

Se logró comprender el funcionamiento interno de componentes relacionados con planificación de procesos, concurrencia y acceso al sistema de archivos. La corrección del bug en `pthread_mutex_trylock` permitió analizar capas de compatibilidad y bibliotecas de hilos, mientras que la implementación del comando `tree` facilitó el uso de llamadas al sistema relacionadas con directorios y archivos.

La modificación del scheduler permitió estudiar políticas de planificación y observar experimentalmente las diferencias entre procesos CPU-bound e interactivos.

Como posible extensión futura, podrían implementarse políticas de scheduling más sofisticadas, soporte adicional para el comando `tree` (flags como `-a`, `-L`), y herramientas de monitoreo de prioridades en tiempo real.

5. Referencias consultadas

- Tanenbaum, A. S. & Woodhull, A. S. *Operating Systems: Design and Implementation*, 3rd ed. Prentice Hall.
 - Documentación oficial de MINIX 3: <https://www.minix3.org>
 - Manual de páginas de MINIX: `man opendir`, `man readdir`, `man lstat`, `man pthread`
-

6. Declaración de uso de IA

Durante el desarrollo del proyecto se utilizaron herramientas de inteligencia artificial generativa como apoyo técnico.

Partes donde se utilizó IA:

- Depuración del bug en `pthread_mutex_trylock`.
- Explicación del funcionamiento del scheduler.
- Asistencia en comandos de compilación y configuración.

Naturaleza del uso:

- Ayuda en depuración de errores.
- Generación y corrección de fragmentos de código.
- Mejora de redacción técnica y estructuración del documento.